COS120 - CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

BÀI 8: BẢNG BĂM

MỤC TIÊU

- Hiểu bảng băm là một cấu trúc dữ liệu dùng để lưu trữ các cặp khóa-giá trị (key,value) cho phép truy cập giá trị nhanh chóng dựa theo khóa.
- Hiểu biết về khái niệm hàm băm, vai trò của nó trong việc **ánh xạ** các khóa tới **chỉ số** trong bảng băm
- Định nghĩa **đụng độ** trong bảng băm và nguyên nhân xảy ra.
- Biết các **kỹ thuật** phổ biến để xử lý đụng độ, bao gồm các phương pháp kết nối và các phương pháp **dò tìm**.

NỘI DUNG

- 8.1 - KHÁI NIỆM VỀ BẢNG BĂM
- 8.2 - XỬ LÝ ĐỤNG ĐỘ TRONG BẢNG BĂM
- 8.3 - HIỆN THỰC BẢNG BĂM

BẢNG BĂM

8.1 - KHÁI NIỆM VỀ BẢNG BĂM

PHÉP BĂM - HASHING

Để hiểu hoạt động của phép băm, ta xem ví dụ đơn giản, cụ thể sau đây. Giả sử ta có các thông tin sau.

- **Đầu vào**: tập hợp các sinh viên (key=mã sinh viên, value=tên sinh viên).
- **Hàm băm:**
  - Gọi H là một hàm băm.
  - Ta chọn H(key) = key%10 (lấy mã sinh viên chia cho 10 lấy dư)
  - Như vậy miền xác định của H(key) là các số dư 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

  Các giá trị này được gọi là chỉ số (index) hay còn gọi là đầu ra.
- **Bảng băm (Hash Table):** Ta tổ chức một bảng gồm hai cột.
  - Cột thứ nhất lưu các chỉ số (index).
  - Cột thứ hai chứa các value.

PHÉP BĂM - HASHING

- Thực hiện phép băm để lưu trữ các sinh viên vào bảng băm

Mã SV Tên SV

KEY	VALUE
17	Bảo
11	Dũng
22	Trình
34	Nhựt
48	Văn
39	Nam
60	Trúc
43	Hùng
15	Toàn
96	Đăng

HÀM BĂM

H(key)=key%10

HASH TABLE

index	value
0	Trúc
1	Dũng
2	Trình
3	Hùng
4	Nhựt
5	Toàn
6	Đăng
7	Bảo
8	Văn
9	Nam

BẢNG BĂM - HASH TABLE

- **Bảng băm (Hash Table):**
  - Là một cấu trúc dữ liệu
  - Để lưu trữ các cặp **khóa-giá trị** (key-value pairs)
  - Cung cấp khả năng truy cập nhanh chóng đến **giá trị** (value) dựa trên **khóa** (key).
- **Các thành phần chính của bảng băm:**
  - Khóa (key): để xác định giá trị đang lưu trữ trong bảng băm.
  - Giá trị (value): là dữ liệu được lưu trữ trong bảng băm, truy cập thông qua khóa.
  - Hàm Băm (Hash Function): nhận vào khóa và trả về một chỉ số (index) trong bảng băm.

CÁC TÁC VỤ TRÊN BẢNG BĂM

- Chèn (Insertion)
- Tìm Kiếm (Search)
- Xóa (Deletion)
- Cập nhật (Update)
- Kiểm tra sự tồn tại (Contains)
- Duyệt (Traversal)

MỘT SỐ ỨNG DỤNG CỦA BẢNG BĂM

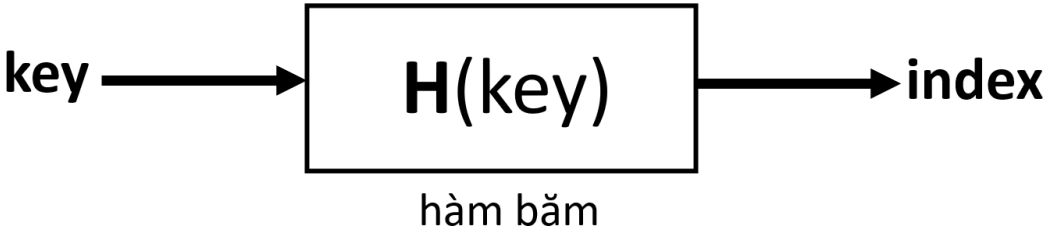
Một số ứng dụng thực tế của bảng băm:

- **Hệ thống quản lý từ điển:** giúp lưu trữ và truy xuất định nghĩa của từ nhanh.
- **Cơ sở dữ liệu:** lập chỉ mục các bản ghi, truy cập dữ liệu nhanh hơn.
- **Bộ nhớ đệm (cache):** lưu trữ dữ liệu tạm thời, giúp tăng tốc truy cập dữ liệu.

Và bảng băm còn nhiều ứng dụng nữa.

HÀM BĂM - HASH FUNCTION

- **Hàm băm (Hash Function):** là một hàm toán học
  - Dùng để **ánh xạ** dữ liệu:
    - từ một không gian lớn (các chuỗi ký tự, các số nguyên,…)
    - vào một không gian nhỏ hơn (các chỉ số).



  - **Input:** là một khóa (key)
  - **Output:** số nguyên index (chỉ đến một vị trí trong bảng băm)
- **Mục tiêu chính của hàm băm**:
  - phân phối các khóa đều đặn và ngẫu nhiên vào không gian chỉ số
  - để giảm thiểu **đụng độ** (collisions).

TÍNH CHẤT CỦA MỘT HÀM BĂM TỐT

- **Tính xác định (Deterministic):** luôn trả về **cùng một chỉ số** cho cùng một khóa.
- **Phân phối đồng đều (Uniform distribution):** phân phối các khóa đồng đều trên không gian chỉ số để giảm thiểu đụng độ.
- **Tính hiệu quả (Efficiency):** phải tính toán nhanh chóng, sử dụng ít tài nguyên.
- **Tính giảm thiểu đụng độ (Minimizing Collisions):** phải giảm thiểu khả năng hai khóa khác nhau được ánh xạ tới cùng một chỉ số.
- **Tính đơn giản (Simplicity):** phải dễ hiểu và dễ hiện thực.
- **Tính bảo mật (Security):** đối với các ứng dụng đòi hỏi bảo mật cao thì hàm băm phải khó đoán và chống lại các tấn công mã hóa.

MỘT SỐ HÀM BĂM ĐƠN GIẢN

- **Division hash function (Hàm băm chia)**

  Công thức: Hash(key) = key % table_size

- **Multiplication hash function (Hàm băm nhân)**

  Công thức: Hash(key) = floor(table_size * (key * A % 1))

- **Simple string hash function (Hàm băm chuỗi đơn giản)**

  Công thức: Hash(key) = (sum of ASCII values of characters in key) % table_size

- **Hàm băm DJB2**

  Công thức:

```
hash = 5381;
hash = ((hash << 5) + hash) + c; // hash * 33 + c
```

BẢNG BĂM

8.2 - XỬ LÝ ĐỤNG ĐỘ TRONG BẢNG BĂM

CÁC NGUYÊN NHÂN ĐỤNG ĐỘ

Nguyên nhân chính dẫn đến đụng độ:

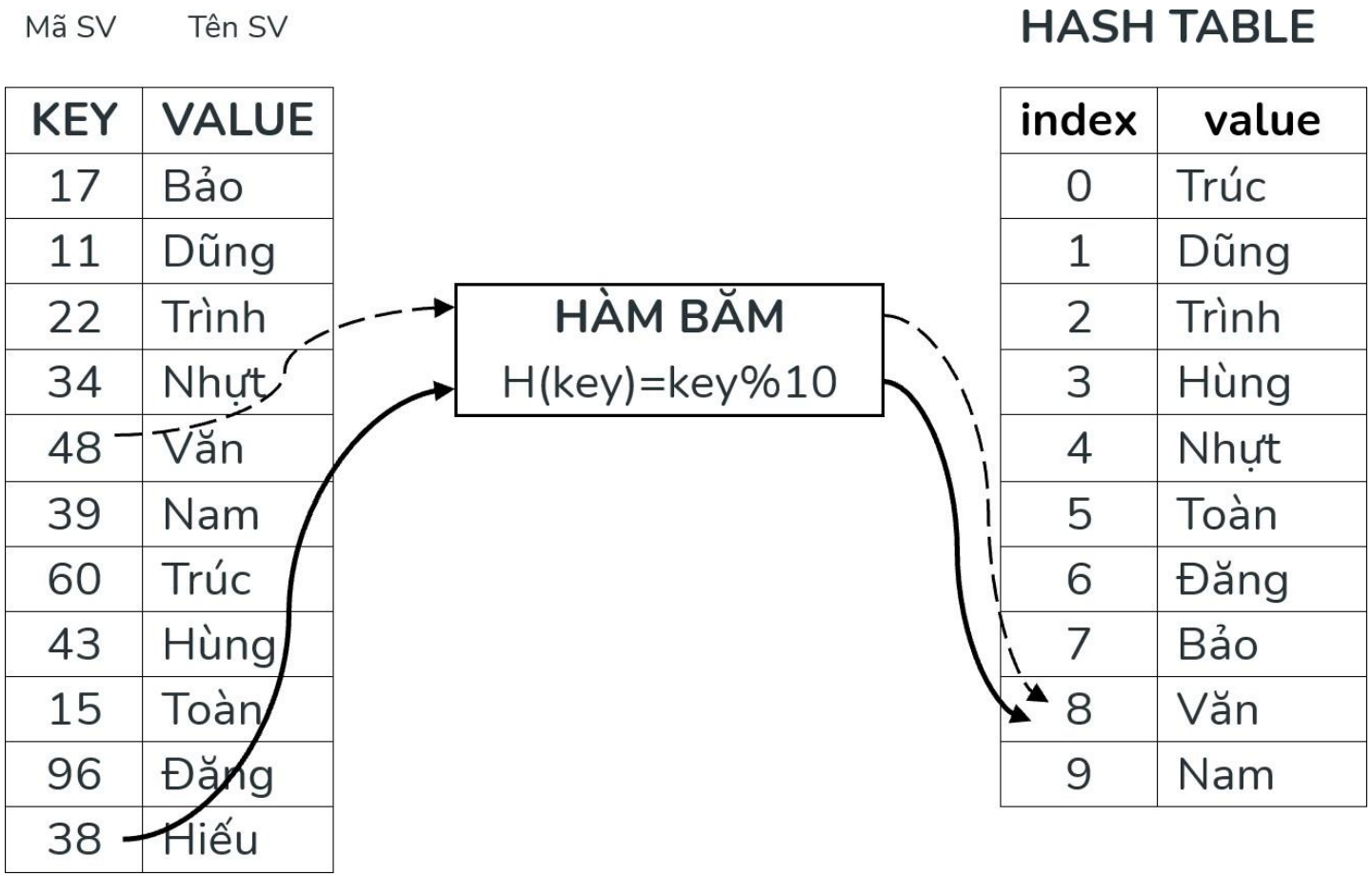
- **Kích thước bảng băm nhỏ**
  - Khi bảng băm **có ít ô chứa** hơn so với số lượng khóa
  - Thì xác suất đụng độ tăng lên.
- **Hàm băm không tốt:**
  - Nếu hàm băm không phân phối các khóa đều đặn
  - Một số chỉ số sẽ tương ứng với nhiều khóa hơn, dẫn đến đụng độ.

Có hai loại đụng độ chính trong bảng băm:

1. **Đụng độ sơ cấp (Primary Collision)**
2. **Đụng độ thứ cấp (Secondary Collision)**

ĐỤNG ĐỘ SƠ CẤP (PRIMARY COLLISION)

- Đụng **độ sơ cấp** xảy ra khi: hai khóa khác nhau được ánh xạ tới **cùng một chỉ số**



ĐỤNG ĐỘ THỨ CẤP (SECONDARY COLLISION)

- **Đụng độ thứ cấp xảy ra khi:**
  - Hai khóa khác nhau đã được xử lý đụng độ sơ cấp.
  - Lại tiếp tục bị động độ lần nữa
- **Ví dụ:**
  - Nếu cả hai khóa key1 và key2 ban đầu được ánh xạ tới vị trí I
  - Và khi sử dụng phương pháp xử lý đụng độ như **double hashing**
  - Cả hai khóa lại được ánh xạ tới **cùng một chỉ số khác sau khi băm lại.**

CÁC PHƯƠNG PHÁP XỬ LÝ ĐỤNG ĐỘ

- **Kết nối riêng biệt (Separate Chaining)**
  - **Mỗi vị trí** trong bảng băm chứa một danh sách liên kết các phần tử có **cùng chỉ số băm**.
  - Khi xảy ra đụng độ, phần tử **mới** được thêm vào danh sách liên kết tại vị trí đó.
- **Kết nối không thay thế (Chaining without replacement)**
  - Khi xảy ra xung đột, cặp khóa-giá trị mới sẽ được **chèn vào khe trống tiếp theo**
  - Mà không làm thay đổi các khóa hiện có.
- **Kết nối có thay thế (Chaining with replacement)**
  - Khi xảy ra xung đột, cặp khóa-giá trị mới sẽ thay thế khóa hiện tại **ở cùng một khe** trong bảng băm.
  - Điều này có nghĩa là khi một khóa mới được thêm vào một khe đã có sẵn các khóa khác, khóa cũ sẽ bị thay thế bởi khóa mới.
- **Dò tuyến tính (Linear Probing):**
  - Khi xảy ra đụng độ, dò tìm ô trống tiếp theo trong bảng băm
  - Dò bằng cách di chuyển tuần tự từ vị trí bị đụng độ.

PHƯƠNG PHÁP KẾT NỐI RIÊNG BIỆT

Phép băm này gồm có:

- Tập dữ liệu đầu vào cần lưu trữ

  {50, 55, 46, 65, 42, 32, 15}
- Hàm băm: H(key)=key%10
- Bảng băm có 10 dòng từ dòng index=0 đến dòng index=9.

BẢNG BĂM

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Bảng băm ban đầu rỗng

Các phần tử thuộc tập dữ liệu đầu vào khi đi qua hàm băm sẽ hình thành tập chỉ mục (key→index) như sau:

key	index
50	0
55	5
46	6
65	5
42	2
32	2
15	5

PHƯƠNG PHÁP KẾT NỐI RIÊNG BIỆT

- Cứ mỗi lần xảy ra đụng độ tại dòng có index nào thì phương pháp này sẽ lưu dữ liệu mới vào ngay sau các dữ liệu đang có tại dòng đó.

BẢNG BĂM

0	50
1	
2	
3	
4	
5	55
6	46
7	
8	
9	

Chèn 50, 55, 46 vào.
Chưa xảy ra xung đột

BẢNG BĂM

0	50	
1		
2		
3		
4		
5	55	65
6	46	
7		
8		
9		

Chèn 65 vào, xung đột tại index 5
Nối 65 vào đuôi của 55.

BẢNG BĂM

0	50	
1		
2	42	
3		
4		
5	55	65
6	46	
7		
8		
9		

Chèn 42 vào, không xung đột.

PHƯƠNG PHÁP KẾT NỐI RIÊNG BIỆT

BẢNG BĂM

0	50	
1		
2	42	32
3		
4		
5	55	65
6	46	
7		
8		
9		

Chèn 32 vào, xung đột tại index 2
Nối 32 vào đuôi của 42.

BẢNG BĂM

0	50		
1			
2	42	32	
3			
4			
5	55	65	15
6	46		
7			
8			
9			

Chèn 15 vào, xung đột tại index 5
Nối 15 vào sau 65.

- Tại mỗi dòng index trong bảng băm, ta có thể dùng danh sách liền kề hoặc danh sách liên kết đơn để lưu trữ các giá trị bị **đụng độ tại dòng đó**. Thậm chí ta cũng có thể dùng cây nhị phân tìm kiếm để lưu trữ dữ liệu cho cho mỗi dòng.

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

- **Bảng băm trong trường hợp này (Chaining without replacement):**
  - Được cài đặt bằng danh sách liền kề, có M nút.
  - Các nút bị xung đột địa chỉ được nối kết với nhau
  - Như một **danh sách liên kết** (nhưng không dùng con trỏ).
- **Mỗi nút của bảng băm là một mẫu tin có hai trường**:
  - Trường value: chứa dữ liệu của nút.
  - Trường next: cho biết vị trí của nút tiếp theo.
- **Khi khởi động bảng băm thì:**
  - Tất cả trường data được gán giá trị là NULLVALUE
  - Tất cả các trường next được gán là -1.

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

Cách giải quyết đụng độ của phương pháp này như sau:

- Khi muốn lưu phần value của cặp key-value (ví dụ cặp x-k) vào bảng băm
  - Khóa k sẽ đi qua hàm băm để sinh ra chỉ số index i.
  - Việc lưu mới, xử lý xung đột cho x diễn ra tại dòng i của bảng băm.
- Nếu chưa bị xung đột thì lưu x tại nút ở dòng i.
- Nếu bị xung đột thì không thể lưu x tại dòng i.
  - Đi tìm nút trống ở phía cuối bảng băm để lưu x vào.
  - Đồng thời cho trường next của nút i chỉ đến vị trí của nút chứa x.

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

Phép băm này gồm có:

{50, 55, 46, 65, 42, 32, 15}

Hàm băm: H(key)=key%10

Bảng băm có M=10 dòng

(index=0 đến index=9)

BẢNG BĂM

0	NULLVALUE	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	NULLVALUE	-1
6	NULLVALUE	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	NULLVALUE	-1

đầu tiên, bảng băm rỗng

Các phần tử thuộc tập dữ liệu đầu vào khi đi qua hàm băm sẽ hình thành tập chỉ mục (key→index) như sau:

key	index
50	0
55	5
46	6
65	5
42	2
32	2
15	5

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

- Khi xảy ra đụng độ tại dòng **index i** thì đi tìm nút trống cuối bảng băm để lưu giá trị mới.
- Ngay sau đó, chuỗi liên kết tại dòng i sẽ kết nạp thêm nút mới làm thành viên.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	-1
6	**46**	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	NULLVALUE	-1

Chèn 50, 55, 46 vào.
Chưa xảy ra xung đột

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	**46**	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	**65**	-1

Muốn thêm 65 vào nhưng đụng độ tại dòng 5. Vì vậy lưu 65 tại dòng 9. Rồi cho next của nút 55 chỉ đến dòng 9.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	**42**	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	**46**	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	**65**	-1

Chèn 42 vào không xảy ra xung đột

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	**42**	8
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	**46**	-1
7	NULLVALUE	-1
8	**32**	-1
9	**65**	-1

Muốn thêm 32 vào nhưng đụng độ tại dòng 2. Vì vậy lưu 32 tại dòng 8. Rồi cho next của nút 42 chỉ đến dòng 8.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	**42**	8
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	**46**	-1
7	**15**	-1
8	**32**	-1
9	**65**	7

Muốn thêm 15 vào nhưng đụng độ tại dòng 5. Vì vậy lưu 15 tại dòng 7. Rồi cho next của nút 65 chỉ đến dòng 7.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	**42**	8
3	**89**	-1
4	**38**	-1
5	**55**	9
6	**46**	-1
7	**15**	3
8	**32**	4
9	**65**	7

- Muốn thêm 38 nhưng đụng 32, cho nên lưu 38 tại dòng 4, cập nhật next của 32 là 4.
- Muốn thêm 89 nhưng đụng 65, cho nên lưu 89 tại dòng 3, cập nhật next của 15 là 3.

PHƯƠNG PHÁP KẾT NỐI CÓ THAY THẾ

- Phương pháp kết nối có thay thế (Chaining with Replacement):
  - Tương tự như PP kết nối không thay thế (Chaining without Replacement).
  - Nhưng nó điểm khác biệt được trình bày như sau.
- Nếu vị trí mà phần tử mới sắp được chèn vào đang bị chiếm dụng bởi một phần tử khác.

Thì xảy ra hai trường hợp sau:

- **TH1:** Nếu phần tử chiếm dụng có khóa trùng với khóa của phần tử mới
  - Thì phần tử mới được chèn vào theo PP Chaining without Replacement như cũ.
- **TH2:** Nếu phần tử chiếm dụng có khóa không trùng với khóa của phần tử mới
  - Thì phần tử chiếm dụng phải dời đi chỗ khác để nhường chỗ cho phần tử mới.
  - Phần tử chiếm dụng được di chuyển đến ô trống gần nhất tính từ cuối bảng băm đi lên.

PHƯƠNG PHÁP KẾT NỐI CÓ THAY THẾ

-Dữ liệu:

{50, 55, 65, 15, 38, 18, 17}

-Hàm băm: H(key)=key%10

-Bảng băm có M=10 dòng (index=0 đến index=9).

BẢNG BĂM

0	NULLVALUE	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	NULLVALUE	-1
6	NULLVALUE	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	NULLVALUE	-1

đầu tiên, bảng băm rỗng

Các phần tử thuộc tập dữ liệu đầu vào khi đi qua hàm băm sẽ hình thành tập chỉ mục (key→index) như sau:

key	index
50	0
55	5
65	5
15	5
38	8
18	8
17	7

PHƯƠNG PHÁP KẾT NỐI CÓ THAY THẾ

- Các bước chèn các dữ liệu vào bảng băm diễn ra như sau.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	-1
6	NULLVALUE	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	NULLVALUE	-1

Chèn 50, 55 vào.
Chưa xảy ra xung đột

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	NULLVALUE	-1
7	NULLVALUE	-1
8	NULLVALUE	-1
9	**65**	-1

Muốn thêm 65 vào nhưng đụng độ tại dòng 5. Vì vậy lưu 65 tại dòng 9. Rồi cho next của nút 55 chỉ đến dòng 9.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	NULLVALUE	-1
7	NULLVALUE	-1
8	**15**	-1
9	**65**	8

Muốn thêm 15 vào nhưng đụng độ tại dòng 5. Vì vậy lưu 15 tại dòng 8. Rồi cho next của nút 65 chỉ đến dòng 8.

PHƯƠNG PHÁP KẾT NỐI CÓ THAY THẾ

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	NULLVALUE	-1
7	**15**	-1
8	**38**	-1
9	**65**	7

Muốn thêm 38 vào nhưng đụng độ tại dòng 8 với nút 15. Vì vậy phải dời 15 đi nhường chỗ cho 38. Sau đó cập nhật lại các giá trị next.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	NULLVALUE	-1
5	**55**	9
6	**18**	-1
7	**15**	-1
8	**38**	6
9	**65**	7

Muốn thêm 18 vào nhưng đụng độ tại dòng 8. Vì vậy lưu 18 tại dòng 6. Rồi cho next của nút 38 chỉ đến dòng 6.

BẢNG BĂM

0	**50**	-1
1	NULLVALUE	-1
2	NULLVALUE	-1
3	NULLVALUE	-1
4	**15**	-1
5	**55**	9
6	**18**	-1
7	**17**	-1
8	**38**	6
9	**65**	4

Muốn thêm 17 vào nhưng đụng độ tại dòng 7 với nút 15. Vì vậy phải dời 15 đi nhường chỗ cho 17. Sau đó cập nhật lại các giá trị next.

PHƯƠNG PHÁP DÒ TUYẾN TÍNH

- **Bảng băm trong trường hợp này:**
  - Được cài đặt bằng một danh sách kề có M nút
  - Mỗi nút có trường key để chứa khoá của nút.

Khi thêm giá trị: khi thêm nút mới có khoá key vào bảng băm, hàm băm H(key) sẽ xác giá trị index i để lưu khóa vào bảng băm tại dòng i.

  - Nếu chưa bị xung đột thì thêm nút mới tại địa chỉ i này.
  - Nếu bị xung đột thì:
    - Dò từ vị trí đụng độ hướng xuống để tìm vị trí trống gần nhất để mà lưu nút mới.
    - Nếu di chuyển đến vị trí M-1 mà chưa thấy chỗ trống thì quay lại vị trí 0 để mà tiếp tục.

PHƯƠNG PHÁP DÒ TUYẾN TÍNH

{50, 55, 46, 65, 42, 32, 25, 15, 98}

Hàm băm: H(key)=key%10

Bảng băm có M=10 dòng

(index=0 đến index=9)

BẢNG BĂM

0	NULLVALUE
1	NULLVALUE
2	NULLVALUE
3	NULLVALUE
4	NULLVALUE
5	NULLVALUE
6	NULLVALUE
7	NULLVALUE
8	NULLVALUE
9	NULLVALUE

đầu tiên, bảng băm rỗng

Các phần tử thuộc tập dữ liệu đầu vào khi đi qua hàm băm sẽ hình thành tập chỉ mục (key→index) như sau:

key	index
50	**0**
55	**5**
46	**6**
65	**5**
42	**2**
32	**2**
25	**5**
15	**5**
98	**8**

PHƯƠNG PHÁP DÒ TUYẾN TÍNH

- Các bước chèn các dữ liệu vào bảng băm diễn ra theo các hình vẽ sau đây.

BẢNG BĂM

0	**50**
1	NULLVALUE
2	NULLVALUE
3	NULLVALUE
4	NULLVALUE
5	**55**
6	**46**
7	NULLVALUE
8	NULLVALUE
9	NULLVALUE

Chèn 50, 55, 46 vào.
Chưa xảy ra xung đột

BẢNG BĂM

0	**50**
1	NULLVALUE
2	NULLVALUE
3	NULLVALUE
4	NULLVALUE
5	**55**
6	**46**
7	**65**
8	NULLVALUE
9	NULLVALUE

Muốn thêm 65 vào nhưng đụng độ tại dòng 5. Vì vậy dò từ dòng 5 xuống. Thấy dòng 7 trống, lưu 65 vào.

BẢNG BĂM

0	**50**
1	NULLVALUE
2	**42**
3	NULLVALUE
4	NULLVALUE
5	**55**
6	**46**
7	**65**
8	NULLVALUE
9	NULLVALUE

Chèn 42 vào.
Không xảy ra xung đột

PHƯƠNG PHÁP DÒ TUYẾN TÍNH

BẢNG BĂM

0	50
1	NULLVALUE
2	42
3	32
4	NULLVALUE
5	55
6	46
7	65
8	NULLVALUE
9	NULLVALUE

Muốn thêm 32 vào nhưng đụng độ tại dòng 2. Vì vậy dò từ dòng 2 xuống. Thấy dòng 3 trống, lưu 32 vào.

BẢNG BĂM

0	50
1	NULLVALUE
2	42
3	32
4	NULLVALUE
5	55
6	46
7	65
8	25
9	15

Muốn thêm 25, rồi lưu 15 vào nhưng đều đụng độ tại dòng 5. Vì vậy dò từ dòng 5 xuống. Thấy dòng 8 và 9 trống, lưu chúng vào.

BẢNG BĂM

0	50
1	98
2	42
3	32
4	NULLVALUE
5	55
6	46
7	65
8	25
9	15

Muốn thêm 98 vào nhưng đụng độ tại dòng 8. Vì vậy dò từ dòng 8 xuống không thấy dòng trống nên quay lên dòng 0 dò tiếp. Dòng 1 trống, lưu 98 vào.

BẢNG BĂM

8.3 - HIỆN THỰC BẢNG BĂM

PHƯƠNG PHÁP KẾT NỐI RIÊNG BIỆT

- Khai báo các cấu dữ liệu và hàm băm.

```
#include <stdio.h>
#include <stdlib.h>

#define M 10
struct node {
    int key;
    struct node *pNext;
};
typedef struct node NODE;

//mỗi bucket[i] để lưu trữ các dữ liệu tại dòng i của hash table
NODE* bucket[M];

int HashFunc(int key) {
    return (key % M);
}
```

PHƯƠNG PHÁP KẾT NỐI RIÊNG BIỆT

Các tác vụ cần cài đặt:

- Hàm tạo nút và hàm kiểm tra rỗng của một dòng
- Hàm chèn giá trị vào bảng băm
- Hàm tìm kiếm
- Hàm xóa
- Hàm cập nhật
- Hàm in nội dung toàn bộ một bảng băm
- Hàm main

(xem code trong giáo trình)

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

- Khai báo các cấu dữ liệu và hàm băm

```
#define M 10
#define NULLVALUE -9999

typedef struct node {
    int key;
    int next;
} NODE;

NODE HashTable[M];
int avail; //cho biết vị trí nút trống

// Hàm băm
int HashFunc(int key) {
    return key % M;
}
```

PHƯƠNG PHÁP KẾT NỐI KHÔNG THAY THẾ

Các tác vụ cần cài đặt:

- Hàm khởi động bảng băm
- Hàm tìm vị trí trống
- Hàm chèn một phần tử vào bảng băm
- Hàm tìm kiếm
- Hàm cập nhật
- Hàm xóa một phần tử
- Hàm in nội dung bảng băm
- Hàm main

(Xem code trong giáo trình)

PHƯƠNG PHÁP KẾT NỐI CÓ THAY THẾ

- Chúng ta đã biết viết code cài đặt bảng băm có giải quyết đụng độ bằng PP thứ nhất "kết nối không thay thế".
- Phương pháp "kết nối có thay thế" này có khác biệt một chút so với PP thứ nhất.
- Vì thế, có thể sửa đổi lại code của phương pháp thứ nhất để có được chương trình máy tính cho PP hai này.

PHƯƠNG PHÁP DÒ TUYẾN TÍNH

- Khai báo các cấu dữ liệu và hàm băm

```
#define M 10
#define NULLVALUE -1
typedef struct {
    int key;
} HashTableEntry;
HashTableEntry HashTable[M];

//Hàm băm
int HashFunc(int key) {
    return key % M;
}
```

PHƯƠNG PHÁP DÒ TUYẾN TÍNH

Các tác vụ cần cài đặt:

- Hàm khởi động bảng băm
- Hàm chèn một phần tử vào bảng băm
- Hàm tìm kiếm
- Hàm xóa một phần tử
- Hàm cập nhật
- Hàm in nội dung bảng băm
- Hàm main

(Xem code trong giáo trình)

BÀI TẬP

BÀI 1: Viết một chương trình hiện thực từ điển Anh-Việt, chương trình có cài đặt bảng băm với phương pháp kết nối riêng biệt. Mỗi nút của bảng băm có khai báo các trường sau:

- Trường word là khoá chứa một từ tiếng anh.
- Trường mean là nghĩa tiếng Việt.
- Trường next là con trỏ chỉ nút kế khi bị xung đột cùng địa chỉ.

Tập khoá là một chuỗi Tiếng Anh, tập địa chỉ có 26 chữ cái. Chọn hàm băm sao cho khoá bắt đầu bằng ký tự a được băm vào địa chỉ 0, b băm vào địa chỉ 1,… cho đến z băm vào địa chỉ 25. Chương trình có những chức năng như sau:

a) Nhập vào một từ

b) Xem từ điển theo ký tự đầu.

c) Xem toàn bộ từ điển.

d) Tra từ điển.

e) Xoá một từ.

f) Xóa toàn bộ từ điển.

g) Thoát khỏi chương trình.

BÀI TẬP

BÀI 2: Viết chương trình xem thông tin sinh viên qua mã số sinh viên.

Thông tin về tất cả sinh viên chứa trong tập tin văn bản, mỗi sinh viên chiếm một dòng văn bản gồm MSSV, họ, tên và điểm các môn học. Chương trình sẽ đọc tập tin văn bản này để tạo ra bảng băm. Mỗi nút của bảng băm bao gồm trường MSSV dùng làm khoá, trường line cho biết vị trí dòng văn bản của sinh viên. Khi truy xuất thông tin về một sinh viên chúng ta nhập MSSV, qua bảng băm chúng ta xác định được vị trí dòng văn bản và đọc thông tin sinh viên qua dòng văn bản này.

Chương trình có các chức năng như sau:

a) Xem tất cả MSSV

b) Xem MSSV trong khoảng từ n1 đến n2 (n1, n2 là số nguyên)

c) Xem thông tin về sinh viên qua MSSV

BÀI 3: Viết code hiện thực bảng băm bằng phương pháp xử lý xung đột kết nối có thay thế (chaining with replacement).

BÀI 4: Viết code hiện thực bảng băm bằng phương pháp xử lý xung đột kết nối riêng biệt. Trong đó, mỗi index của bảng băm tương ứng một cây BST để lưu trữ dữ liệu cho các dữ liệu cùng khóa.

Q&A!